SCHOOL OF COMPUTING & DATA SCIENCE

CAMPUS LOST AND FOUND PORTAL

CampusFind

A Project Report Submitted in Partial Fulfillment of the Requirements
for the Award of the Degree of

BACHELOR OF TECHNOLOGY

in Computer Science and Engineering

Submitted by: Poorna Chandra Manupati (PRN 250200322)

Under Guidance of: Rupam Sah (Course Lecturer)

Institution: Sai University, One Hub Chennai, OMR, Tamil Nadu

Academic Session: 2026–2027 | Semester 3

Course: Web Technology and Internet Programming

Date of Submission: 30 November 2026

Individual Project

BONAFIDE CERTIFICATE

This is to certify that the project report titled “CAMPUS LOST AND FOUND PORTAL” submitted by Poorna Chandra Manupati (PRN 250200322) is a record of bonafide work carried out under my supervision and guidance in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering at Sai University, Chennai.

Project Guide	**Head of Department**	**External Examiner**
Rupam Sah	[HOD name]	[Examiner name]
Signature: _____	Signature: _____	Signature: _____

Date: _____
Place: Chennai

DECLARATION OF ORIGINALITY

I hereby declare that the project report entitled “CAMPUS LOST AND FOUND PORTAL” is submitted as my individual project for Web Technology and Internet Programming at Sai University. The report identifies the external libraries, documentation and AI assistance used in preparing the implementation and supporting material.

I understand that the final submission must represent work I have reviewed, understood and am able to explain and modify during the individual demonstration and viva. I will acknowledge secondary sources and comply with the University’s requirements concerning originality and AI assistance. The AI-use disclosure is provided in Appendix B.

The candidate and authorized faculty must sign the relevant declaration and certificate fields before submission.

Date: 30/11/2026
Place: Chennai

Signature of Candidate: _____

Name: Poorna Chandra Manupati
PRN number: 250200322

ACKNOWLEDGEMENT

I acknowledge Sai University and the School of Computing and Data Science for the academic framework within which this individual project was developed. I acknowledge Rupam Sah, Course Lecturer, and the School faculty as the academic evaluators of this project. The official documentation of the technologies used in the project is acknowledged in the references. AI assistance in design, implementation, testing and writing is disclosed separately in Appendix B.

ABSTRACT

Students who lose belongings on campus often depend on informal announcements, noticeboards and messages that are difficult to search or track. CampusFind addresses this problem through a responsive, database-driven Campus Lost and Found Portal for the Sai University community. The project provides a shared noticeboard where users can report lost or found items, retrieve relevant listings, submit private identification evidence and follow recovery decisions.

The application uses semantic HTML5, responsive CSS and JavaScript ES modules in the browser. A Node.js and Express backend exposes REST APIs, while MongoDB stores users, reports, uploaded images, claims and sessions. Registration and login establish cookie-based sessions. Ownership and administrator checks restrict report changes and claim review. A transaction-based approval workflow marks an item recovered, approves the selected request and rejects competing pending requests together. Possible-match suggestions compare opposite listing types using category, location, keywords and date proximity; they assist discovery without claiming to verify identity.

Evaluation uses an isolated MongoDB replica set, automated API and DOM behaviour tests, and a Postman collection executed with Newman. The recorded evaluation passed 33 automated tests and 29 live Postman assertions across 26 requests. Twelve sample reports were seeded. The application was deployed to Render with MongoDB Atlas, and database health, sample retrieval, persistence across redeployment, public search, item details and matching suggestions were verified. Screenshots document these public views.

Live testing passed 36 workflow checks and 12 separate administrator and student checks, including image persistence, claim decisions and logout. Mobile-device verification remains pending. The results support the tested recovery workflow. Final ownership verification and safe handover remain human responsibilities.

Keywords: Lost and found portal, Express, MongoDB, REST API, claim management

LIST OF ABBREVIATIONS & SYMBOLS

Abbreviation	Expansion or description
API	Application Programming Interface
CRUD	Create, Read, Update and Delete
CSRF	Cross-Site Request Forgery
DOM	Document Object Model
HTTPS	Hypertext Transfer Protocol Secure
REST	Representational State Transfer
TLS	Transport Layer Security
UI	User Interface
XSS	Cross-Site Scripting

LIST OF FIGURES

Figure	Description	Page
3.1	Three-layer application architecture	3
C.1	Live home noticeboard	14
C.2	Live keyword search	15
C.3	Verified recovered-item details	16

LIST OF TABLES

Table	Description	Page
2.1	Comparison of recovery approaches	2
3.1	Database collections and relationships	4
3.2	Application modules	5
4.1	Development tools and libraries	6
5.1	Representative API test cases	8
5.2	Recorded evaluation results	9
A.1	Software and hosting specifications	12

TABLE OF CONTENTS

No.	Topic or chapter title	Page
—	Title page	i
—	Bonafide certificate	ii
—	Declaration of originality	iii
—	Acknowledgement and abstract	iv
—	Abbreviations, figures and tables	v
1	INTRODUCTION	1
1.1–1.4	Background, problem, objectives and report organization	1
2	LITERATURE REVIEW	2
2.1–2.3	Foundations, existing approaches and comparative analysis	2
3	SYSTEM ARCHITECTURE & DESIGN	3
3.1–3.2	Methodology and high-level system design	3
3.3	Database and data schema design	4
3.4–3.5	Module description and matching algorithm	5
4	IMPLEMENTATION DETAILS	6
4.1–4.2	Tools, libraries and core modules	6
4.3–4.4	Security, errors and deployment	7
5	RESULTS & EVALUATION	8
5.1	Experimental setup and test cases	8
5.2	Performance analysis and verification limits	9
6	CONCLUSION & FUTURE SCOPE	10
6.1–6.2	Contributions, limitations and recommendations	10
—	REFERENCES	11
A	Hardware and software specifications	12
B	AI disclosure and individual demonstration	13
C	Application screenshots	14–16

CHAPTER 1: INTRODUCTION

1.1 Project Background & Motivation

Campus belongings move between lecture rooms, the library, dining spaces, residences and sports facilities. A person who finds an item may have no reliable way to identify its owner, while a student who loses it may not know which channel to search. Fragmented reports make duplicate announcements, outdated status information and unverifiable ownership claims more likely.

A campus portal can bring these reports into one searchable workflow. The motivation for CampusFind is to make discovery and recovery visible while preserving private details that help verify ownership. The project also provides a practical setting for applying browser programming, REST APIs, middleware, authentication and database operations together.

1.2 Problem Statement

Develop a responsive full-stack Campus Lost and Found Portal in which users can report items, search and filter listings, submit claims and track recovery requests. A functional backend and persistent database are required. Public listings must not expose private claim evidence, and only authorized users may manage reports or decide recovery requests.

1.3 Project Objectives & Scope

- Provide registration, login, logout and protected access for students and administrators.
- Implement report creation, retrieval, editing, status updates and history-preserving deletion rules.
- Support keyword search, categories, campus-building filters and date sorting.
- Allow private recovery evidence, reporter/admin review and consistent approval or rejection.
- Provide student and administrator dashboards, basic statistics and at least ten sample reports.
- Test successful and unsuccessful API cases and deploy the application with persistent storage.

The scope includes one institutional noticeboard and a proposed possible-match feature. Campus SSO, email delivery, automatic identity verification and private messaging are outside this version. The final custom feature remains subject to instructor approval.

1.4 Organization of the Report

Chapter 2 reviews technical foundations and common recovery approaches. Chapter 3 explains architecture and data design. Chapter 4 describes implementation and deployment. Chapter 5 reports testing and its limits. Chapter 6 presents conclusions and future work. References and appendices provide specifications, disclosure and screenshots.

CHAPTER 2: LITERATURE REVIEW

2.1 Theoretical Foundations

The review focuses on documented web-application practices relevant to the implemented portal. Express security guidance emphasizes input validation, secure transport, cookie configuration and protection against abusive requests [1]. These principles inform validation, security headers, cookie-based sessions and rate limiting in CampusFind. The guidance is a technical foundation, not evidence that every production security risk has been eliminated.

MongoDB documents single-document atomicity and multi-document transactions [2]. A recovery decision changes an item and multiple claims, so approval uses a transaction rather than unrelated writes. Postman documents JavaScript response tests and collection-level execution [3]; these capabilities support the API acceptance collection. Mongoose provides schema and validation mechanisms used by the application models [4].

2.2 Survey of Existing Solutions

Common campus recovery approaches include physical noticeboards, informal group messages and general-purpose spreadsheets or forms. The comparison below evaluates their typical capabilities as workflow patterns. It is not a field survey of named institutions or a benchmark of commercial products.

Table 2.1 Comparison of recovery approaches

Approach	Useful capability	Typical limitation
Physical noticeboard	Visible to people nearby	Manual searching and status updates
Group messages	Quick community distribution	Posts become difficult to retrieve and track
Spreadsheet or form	Structured report collection	Ownership review and access rules require additional workflow
CampusFind	Search, authenticated reports, private evidence and tracked decisions	Requires maintenance and human handover verification

2.3 Comparative Analysis & Research Gap

The project addresses a workflow gap between announcing an item and recording a verified recovery. Search alone is insufficient if request evidence is public or any account can approve a claim. CampusFind connects structured reporting, private evidence and restricted review in one system. The proposed matching heuristic reduces manual browsing, but it has not been validated against a labelled real-campus matching dataset. No claim of superior retrieval accuracy or recovery rate is made.

CHAPTER 3: SYSTEM ARCHITECTURE & DESIGN

3.1 Proposed Methodology

The development method maps the assignment requirements to user flows, data models and REST endpoints. User and item schemas are defined before the recovery workflow. The backend applies validation and authorization independently of browser controls. The frontend then retrieves metadata and records dynamically, and the integration suite checks both successful actions and prohibited operations. Deployment verification follows local and CI checks.

3.2 High-Level System Design

The system has a presentation layer, an application layer and a data layer. Express serves both the static frontend and API from one origin. Browser modules use Fetch and async/await to communicate with the API. Middleware loads sessions, checks CSRF tokens, identifies the user and applies access rules before protected operations. MongoDB stores application records, sessions and image buffers.

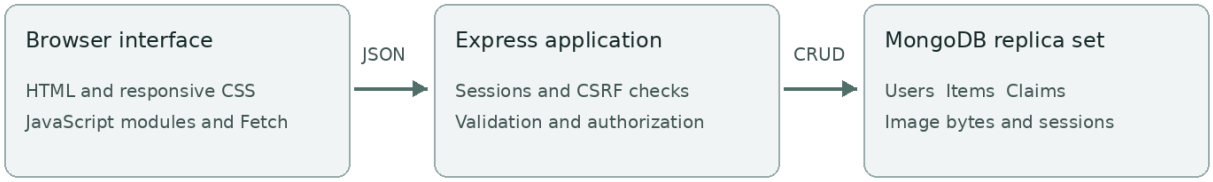


Figure 3.1 Three-layer application architecture

The same-origin design keeps the session cookie and API requests within one application boundary. A public visitor can retrieve reports, metadata, item images and aggregate listing counts. Student access is limited to personal management and authorized claim views. An administrator can manage campus listings and review all requests.

Request and recovery sequence

A browser form validates required fields and sends a request. The API repeats validation, checks ownership or role and writes to MongoDB. A pending claim is private to the parties involved and administrators. When approved, one transaction marks the item recovered, approves the selected request and rejects other pending claims. Failure rolls back those changes together [2].

Design boundaries

The portal coordinates reports and decisions; it does not certify identities or supervise physical exchange. Users should retain unique identifying details for private verification and arrange handover through authorized campus staff. Public descriptions should not contain full identification numbers, financial information or passwords.

3.3 Database & Data Schema Design

References connect users, reports and claims. This avoids embedding an expanding claim history inside each user or item record. Uploaded image data is excluded from routine JSON responses and retrieved through a dedicated endpoint.

Table 3.1 Database collections and relationships

Collection	Principal fields	Relationships and rules
users	name, email, passwordHash, role, timestamps	Unique normalized email; student/admin role
items	name, description, category, location, date, type, status, image, imageType, sample	owner references users; type lost/found; status open/closed/recovered
claims	item, claimant, evidence, status, reviewNote, reviewedBy, timestamps	References item and users; one request per item/claimant pair
sessions	session payload and expiry	Server-side session store; browser holds an opaque identifier

Validation and indexing

The models constrain categories, locations, roles and statuses with enumerations. Name and description lengths are bounded. Report dates must be valid calendar dates and cannot be in the future. Evidence requires at least 15 characters. Uploaded images are limited to one supported file no larger than 3 MB. A unique compound index on item and claimant prevents duplicate requests, including competing inserts.

Listing indexes support recent reports and combinations of type, category, location and status. Keyword searching escapes regular-expression characters before querying the name and description. This is suitable for the small assessment dataset; it is not a replacement for a dedicated indexed search design at larger scale.

Data integrity and lifecycle

Reports with claims cannot be deleted or fully rewritten. They can be closed while retaining history. Recovered items cannot be reopened. Reviewed claims are final in this version. Approval requires an open item and pending request, and runs inside a MongoDB replica-set transaction. Rejecting a request leaves the item available for another verified recovery.

Persistence and sample records

Application records, image bytes and sessions are persisted in Atlas rather than a temporary web-service filesystem. The repeatable seed creates 12 labelled demonstration listings across several categories and locations. Seeded data survived the verified Render redeployment. Re-running the seed preserves existing records and does not reset an existing account password.

3.4 Module Description

Table 3.2 Application modules

Module	Input	Primary responsibility
Accounts	Name, email and password	Register, authenticate, rotate sessions and sign out
Reports	Item fields and optional photo	Validate, persist, retrieve and manage listings
Search	Keyword, type, category, building and date order	Filter and paginate records
Recovery	Private evidence and review decision	Scope evidence and apply claim state transitions
Dashboards	Authenticated identity and role	Show personal activity or administrator statistics
Matching	A selected report	Rank opposite-type candidates in the same category

3.5 Sample Code Implementation / Algorithm

Matching suggestions are an explainable heuristic. Candidate reports must be open, in the same category and of the opposite type. The score uses a matching building, words from the selected item’s name and date proximity. It does not infer identity from private evidence.

```
candidates = open reports with same category and opposite type
words = item.name.lowercase().splitWords().keep(length > 2)
for each candidate:
  score = 3 if candidate.location == item.location else 0
  score += 2 * count(words appearing in candidate name/description)
  score += 1 if absolute date difference < 7 days else 0
return top 4 candidates sorted by descending score
```

For example, a lost white charging adapter can match a found charging adapter in the same category. A nearby date and shared words increase its score. A result with a high score is a suggestion, so the reporter must still inspect private identifying evidence before approval.

Recovery state model

Listing: open → recovered after approved recovery, or open → closed by its reporter/admin. Claim: pending → approved or pending → rejected. The approval transaction also changes other pending claims on that item to rejected. Invalid transitions return a conflict response rather than silently overwriting history.

CHAPTER 4: IMPLEMENTATION DETAILS

4.1 Development Tools & Libraries

Table 4.1 Development tools and libraries

Technology	Use in the project
HTML5 and CSS3	Semantic page structure, forms, layout, responsive rules and focus styles
JavaScript ES modules	DOM rendering, events, Fetch requests and async workflows
Node.js 24 and Express	Application runtime, middleware, REST routes and static serving
MongoDB Atlas and Mongoose	Persistent records, models, validation, indexes and transactions
bcryptjs, express-session and connect-mongo	Password hashing and database-backed sessions
Helmet, express-rate-limit and Multer	Security headers, request limits and bounded photo uploads
Node test runner, Supertest and jsdom	API integration and frontend DOM behaviour checks
Postman, Newman, GitHub Actions and Render	API evidence, CI verification and cloud hosting

4.2 Core Module Implementation

src/models.js defines User, Item and Claim models. src/app.js builds the Express application and contains middleware, validation and API handlers. src/server.js connects the persistent database and starts the HTTP server. src/seed.js provisions sample records without clearing existing data. The frontend separates API/session handling, icons and route rendering into public/api.js, public/icons.js and public/app.js.

The home view retrieves current listing counts and records. A search form updates query parameters, and filtered results are returned in pages of 12. Report forms support JSON fields and multipart photos. Item details retrieve match suggestions. The student dashboard separates personal listings, outgoing requests and incoming requests. The admin view provides campus listings, request review and category statistics.

Fetch responses are checked for errors before rendering. Form submissions disable the submit button while pending and display controlled error messages. Dialogs collect private evidence or review notes. Browser validation improves the form experience; the backend remains responsible for enforcing constraints.

4.3 Security and Error Handling

Registration creates student accounts only. Passwords use salted bcrypt hashes with cost factor 12. Registration requires at least 10 characters and a maximum of 72 UTF-8 bytes. Login and registration regenerate the session identifier and CSRF token. Logout destroys the server session. Production cookies are HTTP-only, same-site and secure. Protected changes require a valid CSRF header in addition to authentication.

An authenticated identity does not automatically authorize an operation. Editing, deleting and reviewing a report require the reporter or an administrator. Claim retrieval is scoped to the claimant and reporter unless the user is admin. Public responses omit password hashes, reporter email and claim evidence. Administrator privileges are provisioned outside public registration.

User text is escaped before HTML insertion. A same-origin Content Security Policy restricts scripts and other resources. API and authentication routes apply rate limits. Upload checks inspect the image byte signature and set the validated image content type. These measures reduce common risks identified in Express guidance [1]; image re-encoding, malware scanning and formal security testing are not implemented.

A central handler returns JSON errors. Typical codes are 400 for invalid data, 401 for anonymous access, 403 for prohibited access or invalid CSRF, 404 for missing records, 409 for duplicates or invalid transitions, and 429 for rate limits. Unexpected server failures return a generic client-facing message.

4.4 Deployment

The source repository is published at https://github.com/poornachandra3217-glitch/Web_Tech_Final_Project. Render hosts the application at https://campusfind-sai.onrender.com using a free web service in Singapore. The confirmed Atlas Cluster0 stores the isolated campusfind database. A dedicated database account has read/write access only to that database and is scoped to the cluster.

Database credentials, session signing material and generated provisioning passwords are stored as private runtime variables. Atlas access was extended only to the service region’s two outbound ranges. The service health endpoint is /api/health. The live startup command runs the repeatable seed before starting Express. Health checks and public retrieval succeeded after redeployment.

An initial 90-second health timeout was resolved by redeploying the same application commit. Subsequent live authentication, image upload, claim approval and rejection checks passed. The timeout cause was not established. Mobile-device checks remain pending; this assessment deployment has no measured availability guarantee.

CHAPTER 5: RESULTS & EVALUATION

5.1 Experimental Setup & Test Cases

Testing uses synthetic data and distinguishes isolated functional checks from live deployment checks. The integration suite starts a real temporary MongoDB replica set, which supports the same transaction mechanism required in Atlas. Supertest sends HTTP requests, while jsdom exercises the actual frontend modules against Express and MongoDB. The Postman collection is executed sequentially with Newman and uses a retained cookie jar and rotating CSRF tokens.

Table 5.1 Representative API test cases from the recorded suite

Case	Action	Expected result	Recorded
T01	Register with invalid email and short password	400 invalid input	Pass
T02	Create report while anonymous	401 protected access	Pass
T03	Create valid item report	201 and stored item	Pass
T04	Retrieve with keyword/type/category/building filters	200 matching items	Pass
T05	Update another user’s report	403 unauthorized	Pass
T06	Update own report	200 and saved fields	Pass
T07	Submit private identifying evidence	201 pending claim	Pass
T08	Repeat the same claim	409 duplicate	Pass
T09	Claimant attempts self-approval	403 unauthorized	Pass
T10	Approve request as reporter	Recovered item and final decisions	Pass
T11	Upload and retrieve a supported image	Matching persisted bytes	Pass
T12	Approve competing claims concurrently	Exactly one winner	Pass

Additional tests cover private evidence visibility, self-claims, CSRF checks, rejected image content, role restrictions, malformed IDs, seed idempotency, logout and frontend rendering/navigation. These cases cover more than the eight API cases required by the assignment. Source tests and retained evidence accompany the report.

Live checks on 8 November 2026 passed 36 workflow checks and 12 separate administrator/student checks. They verified registration, login/logout, uploaded image bytes, filters, private claim scope, rejection, approval and recovered status. The live Postman collection passed 26 requests and 29 assertions. Synthetic records were retained with clear QA labels.

5.2 Performance Analysis

Table 5.2 Recorded evaluation results

Measure	Recorded result	Interpretation
Automated tests	33 passed; 0 failed	28 API and 5 DOM behaviour checks
Postman requests	26 executed; 0 failed	Ordered acceptance workflow in isolated testing
Postman assertions	29 executed; 0 failed	All recorded response assertions passed
Automated suite duration	13.38 seconds	Observed run; includes temporary test setup
Newman mean response time	60.92 ms	Local isolated run, not deployed-user latency
Newman response range	2–457 ms	Small test run; no production percentile estimate
Seeded demonstration data	12 records	6 lost and 6 found reports at verification
Live workflow checks	36 passed; 0 failed	Authentication, reports, uploads, filters and claims
Admin/student smoke	12 passed; 0 failed	Roles, statistics, image retrieval and approval
Live Postman mean / range	791.5 ms / 276–2701 ms	One sequential cloud run; not a load benchmark

The test pass rate demonstrates the checked business rules, not universal correctness. Suite duration includes setup, password hashing and database work, so it should not be interpreted as the application’s response time. Local and live Newman timings describe separate sequential runs; neither establishes throughput under concurrent campus traffic.

Verification limits

No load test, mobile-device test, real-user recovery study or labelled matching-accuracy experiment was completed. The matching feature therefore has no measured precision or recall claim. The free Render service may incur a cold-start delay. The retained results support a functional demonstration within the recorded scope.

Results and screenshots

The live system displays database records dynamically. Appendix C shows the initial noticeboard, keyword search and a verified synthetic recovery. Sanitized HTTP results and a successful workflow screenshot accompany the testing evidence. Authenticated dashboard screenshots remain separate visual work.

Remaining acceptance work

Verify the physical mobile layout, add authenticated dashboard screenshots, obtain instructor approval of the custom feature and complete the required signatures. Submit the report and sanitized testing evidence through Classroom. Live API workflows are verified.

CHAPTER 6: CONCLUSION & FUTURE SCOPE

6.1 Summary of Contributions

CampusFind implements an individual full-stack lost-and-found project using the required frontend and backend technologies with MongoDB persistence. It provides public discovery, protected reporting, scoped recovery evidence, restricted review, personal dashboards and administrator tools. Image bytes and sessions are stored in the database so a web-service redeployment does not erase them.

The recovery transaction is the main consistency mechanism: one approval changes the selected claim, item status and competing pending requests together. Automated tests include concurrent approvals and demonstrate exactly one winning recovery within the tested conditions. Possible-match suggestions add an explainable discovery mechanism while retaining manual ownership checks.

Evaluation passed 33 automated tests, 36 live workflow checks, 12 separate administrator/student checks and 29 assertions across 26 live Postman requests. The Render application uses Atlas persistence. Public browser views and recovered status were verified; physical mobile checks and authenticated dashboard screenshots remain.

6.2 Limitations & Recommendations

- Identity remains human-verified. Future work could integrate institution-managed SSO and verified campus email.
- Account recovery is absent. A production version should add a secure, reviewed password-reset workflow.
- Uploaded photos are signature-checked but not re-encoded or scanned. Stronger media processing and moderation would improve robustness.
- The small-dataset search strategy and latest-200-claims limit need indexed search and pagination at larger scale.
- The heuristic matching score needs evaluation on a labelled campus dataset before accuracy claims or threshold decisions are made.
- Notifications and a supervised handover record could improve follow-through, subject to campus policy.
- A retention policy, monitoring, accessibility testing and load testing should accompany wider campus use.

Before submission, the student should review the implementation, adapt it where needed, complete the evidence, obtain the relevant signatures and be prepared to trace and modify the code during the viva.

REFERENCES

[1] Express. Production best practices for security.

https://expressjs.com/en/advanced/best-practice-security/

[2] MongoDB. Transactions.

https://www.mongodb.com/docs/manual/core/transactions/

[3] Postman. Write scripts to test API response data in Postman.

https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

[4] Mongoose. Validation.

https://mongoosejs.com/docs/validation.html

[5] Express. Session middleware.

https://expressjs.com/en/resources/middleware/session/

[6] Render. Deploying on Render.

https://render.com/docs/deploys

[7] Sai University. Project Report Format.

User-supplied institutional template

[8] CampusFind. Source code, tests and deployment evidence.

https://github.com/poornachandra3217-glitch/Web_Tech_Final_Project

Technical documentation references were consulted for the implementation and report. Template example references to unrelated machine-learning articles are not used as evidence for this web application.

APPENDIX A: HARDWARE & SOFTWARE SPECIFICATIONS

Table A.1 Software and hosting specifications

Component	Specification used or required
Client	Modern browser with JavaScript and Fetch support
Language/runtime	HTML5, CSS3, JavaScript ES modules and Node.js 24
Backend	Express, Mongoose and the dependency versions pinned in package-lock.json
Database	MongoDB Atlas Cluster0; isolated campusfind database; replica-set transactions
Web hosting	Render free Node web service; Singapore region
Testing	Node test runner, Supertest, jsdom, Postman and Newman
Version control and CI	Git, GitHub and GitHub Actions
Local database option	Supplied Docker Compose MongoDB replica set with persistent volume

Key REST endpoints

Accounts: GET /api/auth/me; POST /api/auth/register, /api/auth/login and /api/auth/logout. Reports: GET/POST /api/items; GET/PATCH/DELETE /api/items/:id; GET /api/items/:id/image and /api/items/:id/matches. Recovery: POST /api/items/:id/claims; GET /api/claims; PATCH /api/claims/:id. Admin: GET /api/admin/stats. Health: GET /api/health.

Required runtime configuration

MONGODB_URI and SESSION_SECRET configure persistent storage and sessions. NODE_ENV is production on Render, and PORT is supplied by the host. ADMIN_EMAIL, ADMIN_PASSWORD and DEMO_PASSWORD support the repeatable seed. Secret values are not included in the report, source repository or screenshots.

Submission deliverables

Submit the student-owned GitHub repository, verified deployed URL, report in the prescribed format, API testing evidence and individual demonstration/viva through Google Classroom. The stated deadline is 30 November 2026. The custom feature must be assigned or approved by the instructor.

APPENDIX B: AI USE AND INDIVIDUAL DEMONSTRATION

AI-use disclosure

AI assistance was used to prepare the initial interface design, application code, tests, deployment configuration and documentation. The student must review, understand and adapt the implementation, verify the final results and follow the University's rules for disclosing assistance. This report does not claim unaided authorship or that the student has already completed that review.

Student contribution record

Modules reviewed and understood:

_____

Student changes and rationale:

_____

Tests rerun after student changes:

_____

Instructor approval of custom feature:

_____

Mobile and accessibility checks:

_____

Live upload and recovery verification: passed on 8 November 2026. Evidence: docs/LIVE_TESTING.md and docs/live-testing/.

Individual demonstration sequence

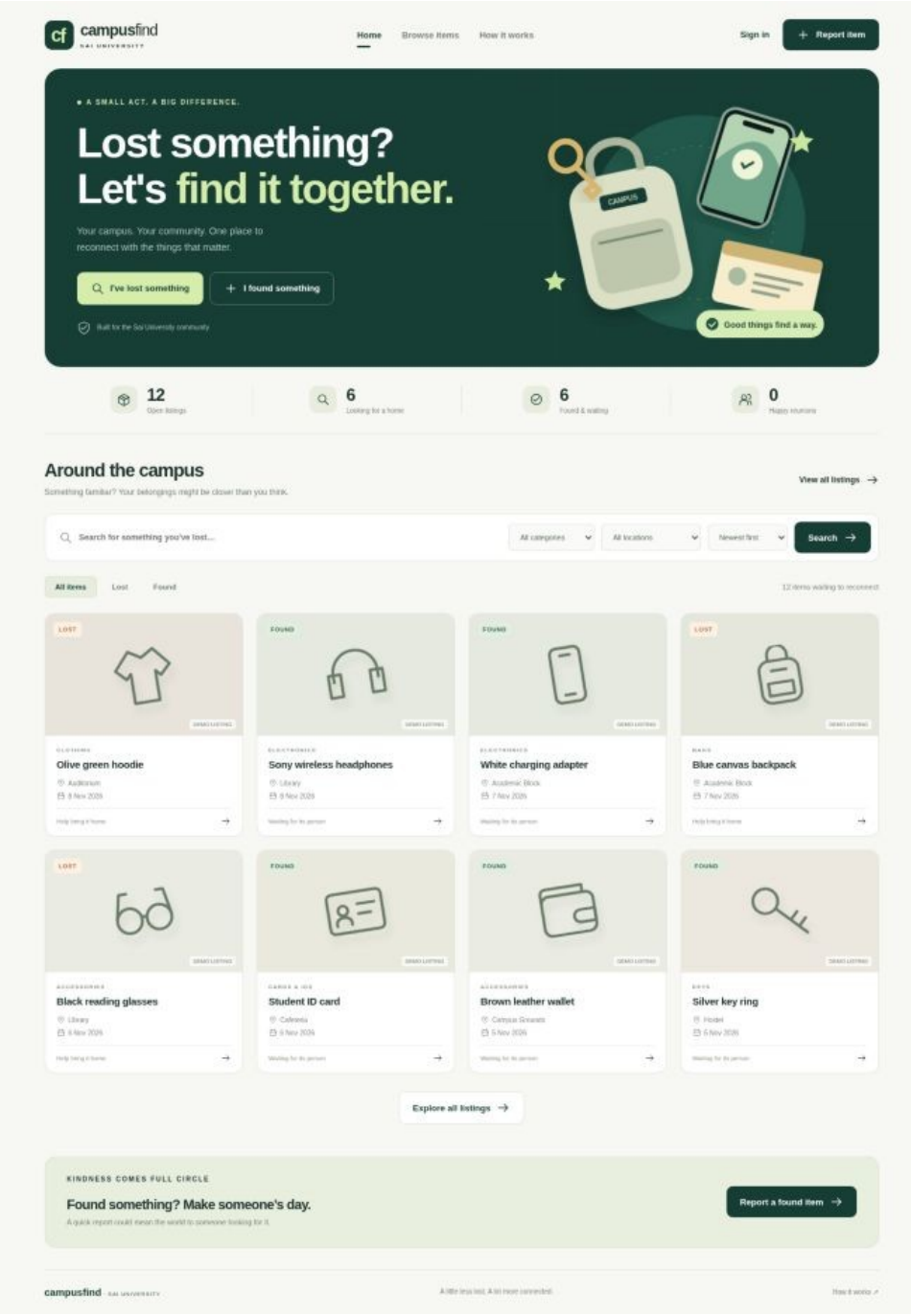
Demonstrate browsing and filtering, registration/login, reporting an item, submitting private evidence as a different user, reviewing the request as the reporter, and checking final status. Explain administrator statistics and one unauthorized API response. Show the Postman collection and recorded results. Trace a request from a DOM event through Fetch, middleware, authorization, a database operation and the response UI.

Viva preparation

Be able to explain authentication versus authorization, CSRF protection, password hashing, the claim compound index, transaction atomicity, image persistence and matching-score limitations. Practise a small change such as adding an approved campus building or adjusting a validation constraint, then run the relevant tests. The student must conduct the demonstration personally.

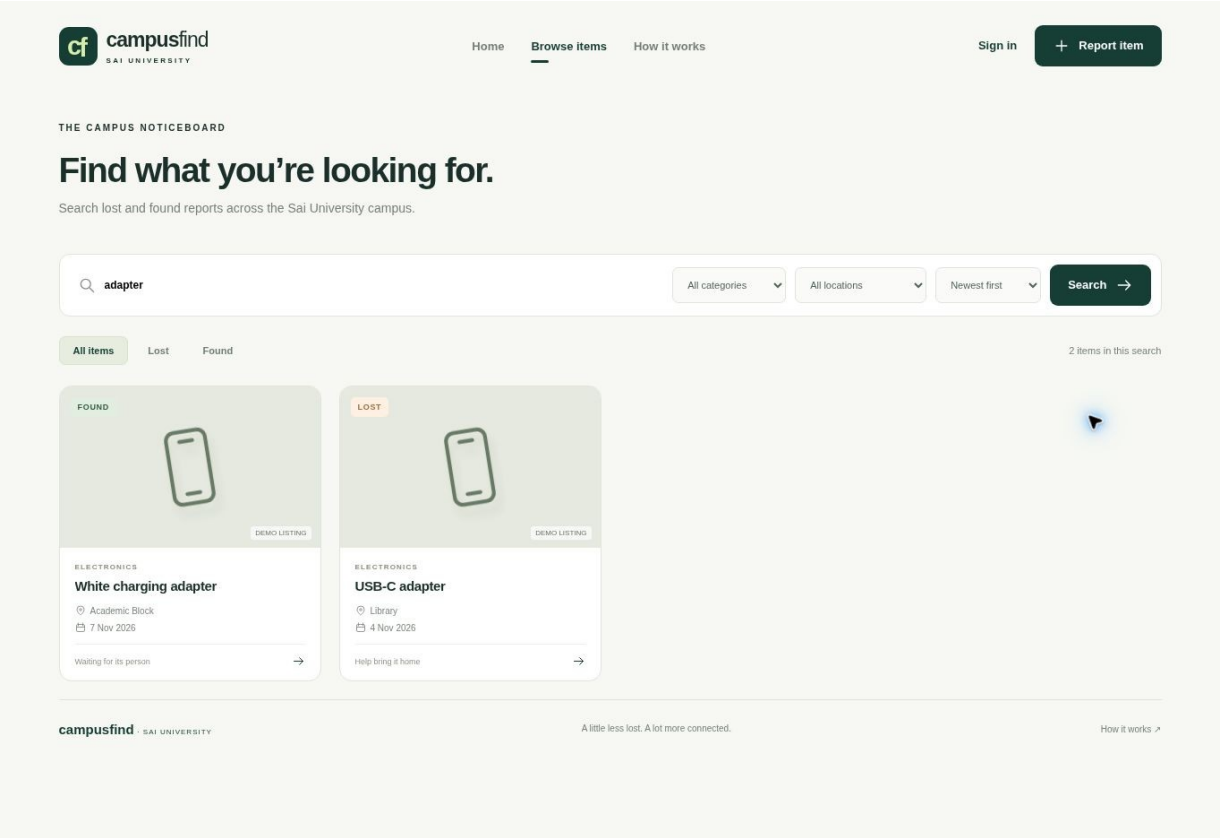
APPENDIX C: APPLICATION SCREENSHOTS

Figure C.1 Live home noticeboard with database-backed sample listings



APPENDIX C: FILTERED SEARCH

Figure C.2 Live keyword search for adapter returns two reports



APPENDIX C: VERIFIED RECOVERY

Figure C.3 Live synthetic item after successful recovery approval

